Part 1: RMS w/ preemption output

```
mhegde — /var/folders/25/93xbx3rs481ggrcm8jrj7z1h0000gn/T/geany_run_script_OI56B1.sh...
                         ----------------------------------------
                         Rate Monotonic Schedule (RMS) Algorithm
                         ----------------------------------------
please input period and execution for A process
default: 25, 10: 25 10
please input period and execution for B process
default: 60, 15: 60 15
CPU Utilization: 0.650000

simulation started
when T=0, process A0 and B0 are generated together
when T=0, program switched to run process A0!
when T=10, process A0 is done
when T=10, program switched to run process B0!
when T=25, process B0 is done
when T=25, process A1 is generated
when T=25, program switched to run process A1!
when T=35, process A1 is done
when T=50, process A2 is generated
when T=50, program switched to run process A2!
when T=60, process A2 is done
when T=60, process B1 is generated
when T=60, program switched to run process B1!
when T=75, process B1 is done
when T=75, process A3 is generated
when T=75, program switched to run process A3!
when T=85, process A3 is done
when T=100, process A4 is generated
when T=100, program switched to run process A4!
when T=110, process A4 is done
when T=120, process B2 is generated
when T=120, program switched to run process B2!
when T=125, process A5 is generated
when T=125, program switched to run process A5!
when T=135, process A5 is done
when T=135, program switched to run process B2!
when T=145, process B2 is done
when T=150, process A6 is generated
when T=150, program switched to run process A6!
when T=160, process A6 is done
when T=175, process A7 is generated
when T=175, program switched to run process A7!
when T=180, process B3 is generated
when T=185, process A7 is done
when T=185, program switched to run process B3!
when T=200, process B3 is done
when T=200, process A8 is generated
when T=200, program switched to run process A8!
```

Part 2: EDF w/ preemption output

```
                        ---------------------------------------
                        Earliest Deadline First (EDF) Algorithm
                        ---------------------------------------
please input period and execution for A process
default: 25, 10: 20 10
please input period and execution for B process
default: 60, 15: 50 25
CPU Utilization: 1.000000

simulation started
when T=0, process A0 and B0 are generated together
when T=0, program switched to run process A0!
when T=10, process A0 is done
when T=10, program switched to run process B0!
when T=20, process A1 is generated
-when T=20, program switched to run process A1!
when T=30, process A1 is done
when T=30, program switched to run process B0!
when T=40, process A2 is generated
when T=45, process B0 is done
when T=45, program switched to run process A2!
when T=50, process B1 is generated
when T=55, process A2 is done
when T=55, program switched to run process B1!
when T=60, process A3 is generated
-when T=60, program switched to run process A3!
when T=70, process A3 is done
when T=70, program switched to run process B1!
when T=80, process A4 is generated
-when T=80, program switched to run process A4!
when T=90, process A4 is done
when T=90, program switched to run process B1!
when T=100, process B1 is done
when T=100, process A5 and B2 are generated together
when T=100, program switched to run process A5!
when T=110, process A5 is done
when T=110, program switched to run process B2!
when T=120, process A6 is generated
-when T=120, program switched to run process A6!
when T=130, process A6 is done
when T=130, program switched to run process B2!
when T=140, process A7 is generated
when T=145, process B2 is done
when T=145, program switched to run process A7!
when T=150, process B3 is generated
when T=155, process A7 is done
when T=155, program switched to run process B3!
when T=160, process A8 is generated
-when T=160, program switched to run process A8!
when T=170, process A8 is done
when T=170, program switched to run process B3!
when T=180, process A9 is generated
-when T=180, program switched to run process A9!
when T=190, process A9 is done
when T=190, program switched to run process B3!
when T=200, process B3 is done
when T=200, process A10 and B4 are generated together
when T=200, program switched to run process A10!
```

Part 1: Multithreading output

```
pi@raspberrypi:~/Documents/Lab4 $ sudo taskset [illegible] pthread_example
Priority of main thread: 99
Creating thread3...
Thread 3 start
Thread 3 requests the lock
Thread 3 had the lock
Thread 3 running, prioirty 1, process 10%...
Creating thread1...
Thread 1 start
Thread 1 requests the lock
Thread 3 running, prioirty 10, process 20%...
Creating thread2...
Thread 3 running, prioirty 10, process 30%...
Thread 3 running, prioirty 10, process 40%...
Thread 3 running, prioirty 10, process 50%...
Thread 3 running, prioirty 10, process 60%...
Thread 3 running, prioirty 10, process 70%...
Thread 3 running, prioirty 10, process 80%...
Thread 3 running, prioirty 10, process 90%...
Thread 3 released the lock
Thread 1 had the lock
Thread 1 running, prioirty 10, process 10%...
Thread 1 running, prioirty 10, process 20%...
Thread 1 running, prioirty 10, process 30%...
Thread 1 running, prioirty 10, process 40%...
Thread 1 running, prioirty 10, process 50%...
Thread 1 running, prioirty 10, process 60%...
Thread 1 running, prioirty 10, process 70%...
Thread 1 running, prioirty 10, process 80%...
Thread 1 running, prioirty 10, process 90%...
Thread 1 released the lock
Thread 1 complete
Thread 2 start
Thread 2 had the lock
Thread 2 running, prioirty 5, process 10%...
Thread 2 running, prioirty 5, process 20%...
Thread 2 running, prioirty 5, process 30%...
Thread 2 running, prioirty 5, process 40%...
Thread 2 running, prioirty 5, process 50%...
Thread 2 running, prioirty 5, process 60%...
Thread 2 running, prioirty 5, process 70%...
Thread 2 running, prioirty 5, process 80%...
Thread 2 running, prioirty 5, process 90%...
Thread 2 complete
Thread 3 complete
pi@raspberrypi:~/Documents/Lab4 $
```

PART 3 Code:

matrixmult.cpp | commands.txt | pthread_example.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#include <sched.h>
#define _GNU_SOURCE

#include <sys/syscall.h>

#ifndef SYS_gettid
#error "SYS_gettid unavailabile on this system"
#endif

#define gettid() ((pid_t)syscall(SYS_gettid))

pthread_mutex_t lock;
pthread_mutexattr_t attributes;

const long int LOOP_LARGE_NUMBER = 500000000;
const long int LOOP_10PERCENT_NUMBER = LOOP_LARGE_NUMBER/10;

int policy = SCHED_FIFO;

int get_real_time_priority() {
    char str[50];
    sprintf(str, "/proc/%d/stat", gettid());
    FILE *fp = fopen(str, "r");
    for (int i = 0; i < 17; i++) {
        fscanf(fp, "%s", str);
    }
    int rt_priority = 0;
    fscanf(fp,"%d",&rt_priority);
    fclose(fp);
    return -rt_priority-1;
}

void *function1() {
    printf("Thread 1 start\n");
    fflush(stdout);

    printf("Thread 1 requests the lock\n");
    fflush(stdout);

    pthread_mutex_lock(&lock);

    printf("Thread 1 had the lock\n");
    fflush(stdout);

    int counter = 0;
    for(int i = 0; i < LOOP_LARGE_NUMBER; i++) {
        if ((i > 0) && (i % LOOP_10PERCENT_NUMBER == 0)) {
            counter += 1;
            printf("Thread 1 running, prioirty %d, process %d0%%...\n", get_real_time_priority(), counter);
        }
    }

    printf("Thread 1 released the lock\n");
    fflush(stdout);

    pthread_mutex_unlock(&lock);
```

col: 0 sel: 230 INS TAB mode: LF encoding: UTF-8 filetype: C scope: unknown

View Document Project Build Tools Help

matrixmult.cpp | commands.txt | pthread_example.c

```c
    printf("Thread 1 complete\n");
    fflush(stdout);
}

void *function2() {
    printf("Thread 2 start\n");
    fflush(stdout);

    //printf("Thread 2 requests the lock\n");
    //fflush(stdout);

    //pthread_mutex_lock(&lock);

    printf("Thread 2 had the lock\n");
    fflush(stdout);

    int counter = 0;
    for(int i = 0; i < LOOP_LARGE_NUMBER; i++) {
        if ((i > 0) && (i % LOOP_10PERCENT_NUMBER == 0)) {
            counter += 1;
            printf("Thread 2 running, prioirty %d, process %d0%%...\n", get_real_time_priority(), counter);
        }
    }

    //printf("Thread 2 released the lock\n");
    //fflush(stdout);

    //pthread_mutex_lock(&lock);

    printf("Thread 2 complete\n");
    fflush(stdout);
}

void *function3() {
    printf("Thread 3 start\n");
    fflush(stdout);

    printf("Thread 3 requests the lock\n");
    fflush(stdout);

    pthread_mutex_lock(&lock);

    printf("Thread 3 had the lock\n");
    fflush(stdout);

    int counter = 0;
    for(int i = 0; i < LOOP_LARGE_NUMBER; i++) {
        if ((i > 0) && (i % LOOP_10PERCENT_NUMBER == 0)) {
            counter += 1;
            printf("Thread 3 running, prioirty %d, process %d0%%...\n", get_real_time_priority(), counter);
        }
    }

    printf("Thread 3 released the lock\n");
    fflush(stdout);

    pthread_mutex_unlock(&lock);

    printf("Thread 3 complete\n");
    fflush(stdout);
```

col: 0 sel: 230 INS TAB mode: LF encoding: UTF-8 filetype: C scope: unknown

matrixmult.cpp | commands.txt | pthread_example.c

```c
}

int main() {
    pthread_mutexattr_setprotocol(&attributes,PTHREAD_PRIO_INHERIT);
    pthread_mutex_init(&lock,&attributes);

    struct sched_param param_main = {0};
    int priority_mainthread = 99;
    param_main.sched_priority = priority_mainthread;
    pthread_setschedparam(pthread_self(), policy, &param_main);
    printf("Priority of main thread: %d\n", get_real_time_priority());

    int priority3 = 1;
    pthread_t thread3 = {0};
    struct sched_param param3 = {0};
    param3.sched_priority = priority3;
    pthread_attr_t attr3;
    pthread_attr_init(&attr3);
    pthread_attr_setinheritsched(&attr3, PTHREAD_EXPLICIT_SCHED);
    pthread_attr_setschedpolicy(&attr3, policy);
    pthread_attr_setschedparam(&attr3, &param3);

    //sleep(1);
    printf("Creating thread3...\n");
    fflush(stdout);
    pthread_create(&thread3, &attr3, function3, NULL);
    sleep(1);




    int priority1 = 10;
    pthread_t thread1 = {0};
    struct sched_param param1 = {0};
    param1.sched_priority = priority1;
    pthread_attr_t attr1;
    pthread_attr_init(&attr1);
    pthread_attr_setinheritsched(&attr1, PTHREAD_EXPLICIT_SCHED);
    pthread_attr_setschedpolicy(&attr1, policy);
    pthread_attr_setschedparam(&attr1, &param1);

    //sleep(1);
    printf("Creating thread1...\n");
    fflush(stdout);
    pthread_create(&thread1, &attr1, function1, NULL);
    sleep(1);

    //pthread_join(thread1, NULL);



    int priority2 = 5;
    pthread_t thread2 = {0};
    struct sched_param param2 = {0};
    param2.sched_priority = priority2;
    pthread_attr_t attr2;
    pthread_attr_init(&attr2);
    pthread_attr_setinheritsched(&attr2, PTHREAD_EXPLICIT_SCHED);
```

ol: 0 sel: 230 INS TAB mode: LF encoding: UTF-8 filetype: C scope: unknown

```c
    //sleep(1);
    printf("Creating thread1...\n");
    fflush(stdout);
    pthread_create(&thread1, &attr1, function1, NULL);
    sleep(1);

    //pthread_join(thread1, NULL);



    int priority2 = 5;
    pthread_t thread2 = {0};
    struct sched_param param2 = {0};
    param2.sched_priority = priority2;
    pthread_attr_t attr2;
    pthread_attr_init(&attr2);
    pthread_attr_setinheritsched(&attr2, PTHREAD_EXPLICIT_SCHED),
    pthread_attr_setschedpolicy(&attr2, policy);
    pthread_attr_setschedparam(&attr2, &param2);

    //sleep(1);
    printf("Creating thread2...\n");
    fflush(stdout);
    pthread_create(&thread2, &attr2, function2, NULL);
    sleep(1);

    //pthread_join(thread2, NULL);




    pthread_join(thread3, NULL);
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    return 0;
}
```

col: 19 sel: 0 INS TAB mode: LF encoding: UTF-8 filetype: C scope: mai